

PROJECT REPORT

EduGenie – AI Powered Learning Assistant

(EduGenie: Google Gemini Powered Learning Assistant)

Team ID	6ab7f31f8c7c66e70672939b
College	Government College of Engineering, Salem
Department	Computer Science and Engineering
Academic Year	2026–2027

Team Members

S.No	Name
1	Mohammed Yasin K
2	Gokulnath S
3	Mathavan A
4	Jayapriyan J
5	Dinesh Kumar M

Project Report Format

1. INTRODUCTION

- 1.1 Project Overview
- 1.2 Purpose

2. IDEATION PHASE

- 2.1 Problem Statement
- 2.2 Empathy Map Canvas
- 2.3 Brainstorming

3. REQUIREMENT ANALYSIS

- 3.1 Customer Journey map
- 3.2 Solution Requirement
- 3.3 Data Flow Diagram
- 3.4 Technology Stack

4. PROJECT DESIGN

- 4.1 Problem Solution Fit
- 4.2 Proposed Solution
- 4.3 Solution Architecture

5. PROJECT PLANNING & SCHEDULING

- 5.1 Project Planning

6. FUNCTIONAL AND PERFORMANCE TESTING

- 6.1 Performance Testing
- 6.2 User Acceptance Testing

7. RESULTS

- 7.1 Output Screenshots

8. ADVANTAGES & DISADVANTAGES

9. CONCLUSION

10. FUTURE SCOPE

11. APPENDIX

- Source Code (if any)
- Dataset Link
- GitHub & Project Demo Link

1. INTRODUCTION

1.1 Project Overview

EduGenie is a web-based, AI-powered learning assistant developed for students and learners. It brings five learning functions into one interface: **Question Answering (Q&A)**, **Concept Explanation**, **Quiz Generation**, **Summarization** and a **Personalized Learning Path**. A student selects a function, types a question, topic or passage (up to 4000 characters), chooses a learner level (Beginner, Intermediate or Advanced) and submits the request.

The front end is built with HTML and CSS and is served through a Jinja2 template. It sends the request to a **FastAPI** backend running on **Uvicorn**. The backend validates the input and routes it to the endpoint of the selected function (/qa, /explain, /quiz, /summarize or /learn/recommendations). A feature-specific Python module defines the instruction and the expected response structure, the AI model generates the response, and the backend returns a structured result that the interface displays as result cards. The application is intended to run locally at <http://127.0.0.1:8000>, and its Gemini-based functions require a valid Google Gemini API key.

Function	What the student can do
Q&A	Ask an academic question and receive a short answer with key points, an example and a next step.
Concept Explanation	Enter a concept and receive a simplified, step-by-step explanation with an example and key takeaways.
Quiz Generation	Enter a topic or material and attempt five multiple-choice questions with feedback.
Summarization	Paste educational text and receive a concise summary, key points and important terms.
Learning Path	Enter a topic and receive a staged learning path with suggested resources.

1.2 Purpose

The purpose of EduGenie is to support self-paced academic learning through a single, simple assistant. Specifically, the system is designed to:

- simplify difficult academic concepts so that they are easier to understand at the selected learner level;
- give quick, concise answers to academic questions and help clear doubts;
- generate multiple-choice quizzes so that students can practise and check their understanding;
- summarize educational text so that long content is quicker to revise;
- provide a structured learning path that shows what to learn next for a given topic.

EduGenie supports learning; its output is AI-generated and should be verified against course material.

2. IDEATION PHASE

2.1 Problem Statement

Students frequently study alone from textbooks, notes and online resources. Three learning difficulties motivate EduGenie: (1) difficulty understanding complex academic concepts and getting doubts clarified quickly; (2) difficulty following a structured learning progression, that is, not knowing what to learn next; and (3) difficulty reviewing long content efficiently and checking one's understanding through practice. The statements below follow the customer problem statement template and are design-oriented; they are not survey findings.

Problem Statement (PS)	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	A student studying academic subjects	understand a difficult concept and clear my doubt	the explanation I have is hard to follow and help is not always immediately available	the material may be written at a level or in language that does not match my current understanding	Confused and uncertain
PS-2	A student who wants to learn a topic in an organised way	know which related topics to study next	I am not sure about the order of topics	self-study resources are many and are not organised for my level	Unguided and unsure of direction
PS-3	A student revising before a test	review long content quickly and check my understanding	revision takes a lot of time and I have no quick way to test myself	content is lengthy and practice questions are not readily available	Overwhelmed and unsure of my readiness

2.2 Empathy Map Canvas

The empathy map captures what a learner using EduGenie may think and feel, hear, see, say and do, together with the pains and gains that the project addresses. It follows the empathy map canvas template and is derived from the project's problem context rather than from measured user research.

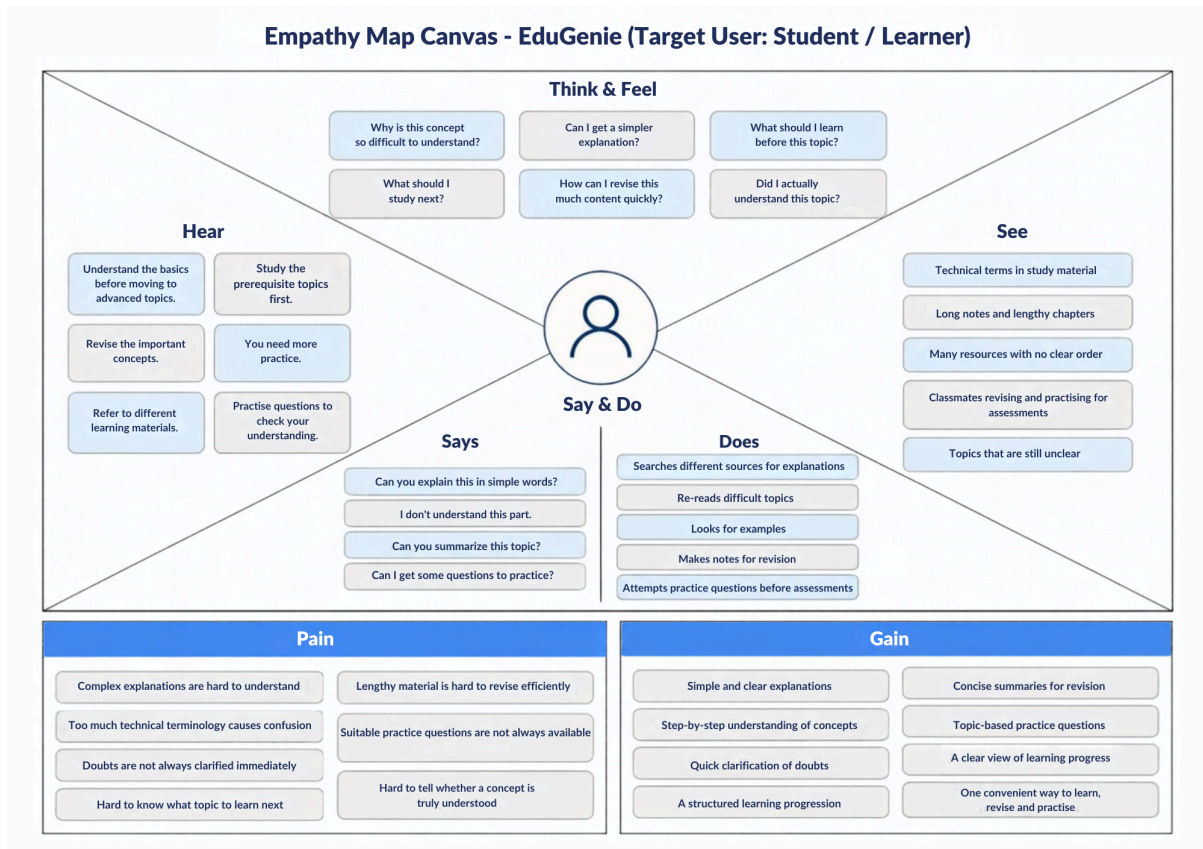


Fig. 2.1 – Empathy map of the EduGenie learner

2.3 Brainstorming

Step 1: Problem statement selected

How might we help students understand, clarify, revise and practise academic content, and know what to learn next, through one simple web-based assistant?

Step 2: Idea listing and grouping

Theme	Ideas
Understanding	Q&A assistant; concept explanation with step-by-step understanding; learner-level selection (Beginner / Intermediate / Advanced)
Revision and practice	Summarization with key points and important terms; five-question multiple-choice quiz with answer feedback
Guidance	Learning path with ordered stages, topics, suggested duration and resources
Interface	Single page with task tabs; result cards; copy, regenerate and new-task controls
Future ideas (not implemented)	Voice interaction; multilingual learning; mobile application; offline capability; progress tracking; gamification; adaptive learning; group study; teacher / parent dashboards; LMS integration; image / PDF input; smart notifications

Step 3: Idea prioritization

The ideas were placed on an importance–feasibility grid. The placement is qualitative and reflects the scope that was actually implemented: the five core learning functions are important and feasible within the project, whereas the future ideas are shown separately and are not part of the current implementation.

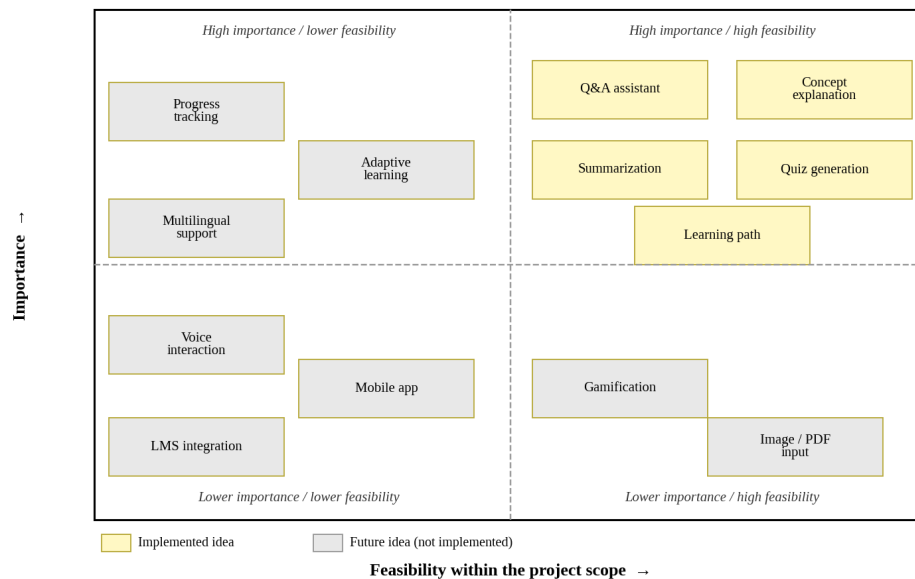


Fig. 2.2 – Idea prioritization (qualitative)

3. REQUIREMENT ANALYSIS

3.1 Customer Journey map

The journey below follows a student who uses EduGenie to learn, revise or practise a topic. It reflects the actual workflow of the interface.

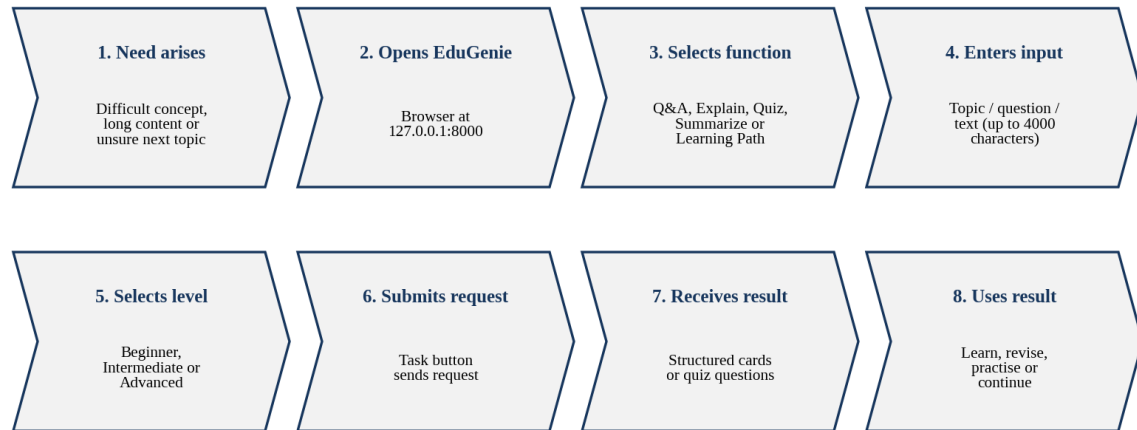


Fig. 3.1 – Student journey with EduGenie

Stage	Student action / experience	Touchpoint	Possible pain point	EduGenie support
Need arises	Meets a difficult concept, long content, or is unsure what to study next	Notes, textbooks	Concept stays unclear; unsure where to start	–
Opens EduGenie and chooses a function	Opens the web page and picks Q&A, Explain, Quiz, Summarize or Learning Path	Web interface, task tabs	May be unsure which function suits the need	Five clearly labelled task tabs on one page
Enters input and level	Types a topic, question or text and selects a learner level	Input box, level selector, character counter	May find it hard to phrase the request	Input guidance text for each task; levels adapt the language
Submits and waits	Presses the task button and waits for the response	Task button	Response time depends on the AI model / service; the service may be busy	Retry and clear error messages (e.g. busy service, invalid input)
Receives result	Reads the explanation, answer, summary, quiz or path	Result cards, quiz options	Output may need verification against academic material	Structured cards; copy, regenerate and new-task controls
Uses the result	Learns, revises, practises or continues with the next topic	Result cards	No saved history of earlier sessions	Next step, key points, quiz feedback and learning-path stages

3.2 Solution Requirement

Functional Requirements

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	Concept Explanation	The student enters a concept and a learner level; the system returns a simplified explanation, step-by-step understanding, an example and key takeaways (/explain).

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-2	Academic Question & Doubt Clarification	The student enters a question; the system returns a concise answer, key points, an example and a next step (/qa).
FR-3	Learning Recommendations	The student enters a topic; the system returns a learning path of ordered stages with topics, suggested duration and resources (/learn/recommendations).
FR-4	Content Summarization	The student pastes content; the system returns a concise summary, key points and important terms (/summarize).
FR-5	Quiz & Knowledge Assessment	The student enters a topic or material; the system returns five multiple-choice questions with four options each; the interface shows correct answers, explanations and a score (/quiz).
FR-6	Learner level and interface	The student can choose Beginner, Intermediate or Advanced, switch between functions, copy a response, regenerate it or start a new task.

Non-functional Requirements

NFR No.	Non-Functional Requirement	Description
NFR-1	Usability	A single, simple page with labelled task tabs, a clear input box and one action button; results are shown as readable cards.
NFR-2	Readability	Output is structured (title, key points, example, steps) and written at the selected learner level.
NFR-3	Modularity & Maintainability	Each function is implemented in its own Python module and is connected to its own endpoint, so a function can be modified without changing the others.
NFR-4	Reliability	Input is validated (non-empty, at most 4000 characters, valid level); requests time out after 45 seconds; temporary service errors (429, 500, 503) are retried; malformed AI output returns a clear error message.
NFR-5	Security	The Gemini API key is read from an environment variable and is not sent to the browser. No user accounts or database are used.
NFR-6	Local Execution & Response Practicality	The application runs locally with Uvicorn. Response speed depends on the AI service; no measured performance guarantees are claimed.

3.3 Data Flow Diagram

The data flow diagram shows how data moves from the student through the interface and backend to the feature module and the AI model, and how the result returns. The system has no database or data store; data flows only between the student, the application processes and the AI service.

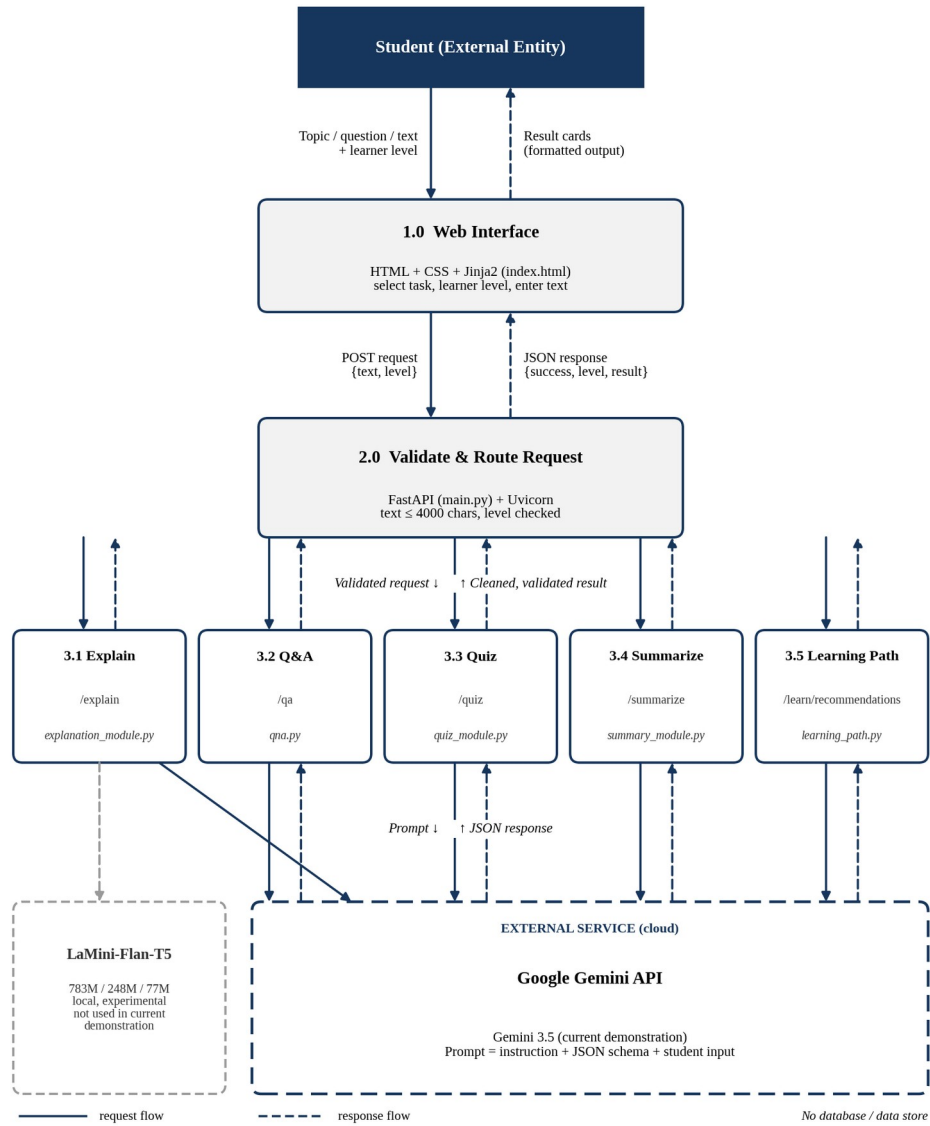


Fig. 3.2 – Data flow diagram of EduGenie

1. The student selects a function, enters the topic, question or text and chooses a learner level in the web interface.
2. The interface sends a POST request containing the text and level to the selected endpoint of the FastAPI backend.
3. FastAPI validates the request (non-empty, up to 4000 characters, valid level) and routes it to the corresponding feature module.
4. The module supplies the instruction and the expected response structure; the backend builds the prompt and sends it to the Google Gemini API.
5. The AI response is returned as JSON, cleaned and checked by the module, and sent back to the interface as {success, level, result}.
6. The interface displays the result as cards (or as an interactive quiz) to the student.

LaMini-Flan-T5 (shown dashed) is the locally tested, experimental model for explanation. It is not used in the current demonstration, in which Concept Explanation uses Gemini 3.5 (see Section 4.2).

3.4 Technology Stack

Table 1: Components & Technologies

S.No	Component	Description	Technology
1	User Interface	Single-page web interface with task tabs, input box, learner-level selector and result cards	HTML, CSS
2	Template Engine	Renders index.html from the FastAPI application	Jinja2
3	Application Logic-1	Request handling, validation, routing, retry and error handling (main.py)	Python 3.10+, FastAPI
4	Application Logic-2	Feature modules: qna.py, explanation_module.py, quiz_module.py, summary_module.py, learning_path.py	Python
5	Application Server	Runs the FastAPI application locally (uvicorn main:app --reload)	Uvicorn
6	Database	Not applicable – no database is used	Not Applicable
7	Cloud Database	Not applicable	Not Applicable
8	File Storage	Templates and static files on the local filesystem	Local filesystem (templates/, static/)
9	External API	Cloud generative AI service used for AI responses	Google Gemini API
10	Machine Learning Model (current)	Generates responses for the learning functions; Gemini 3.5 is used in the current Concept Explanation demonstration	Google Gemini
11	Machine Learning Model (experimental)	Local explanation model tested during development (783M, 248M and 77M variants); not used in the current demonstration	LaMini-Flan-T5
12	Infrastructure	Local execution at http://127.0.0.1:8000; no cloud deployment	Local system

Table 2: Application Characteristics

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Open-source Python web and templating technologies are used for the backend and interface.	FastAPI, Jinja2, Uvicorn, Pydantic
2	Security Implementations	API key kept in an environment variable and never sent to the browser; input validation (length limit, learner-level check). No authentication, encryption or firewall is implemented.	Environment variable, Pydantic validation
3	Scalable Architecture	Frontend, backend and feature modules are separated, which eases future extension. No horizontal scaling or load balancing is implemented.	FastAPI, modular Python
4	Availability	Availability depends on the running local application and, for Gemini functions, on the Google Gemini API. Retries are applied for temporary errors.	Uvicorn, Gemini API
5	Performance	Request timeout of 45 seconds and retry with waiting for temporary errors; response time depends on the model or service. No measured values are available.	FastAPI (asynchronous), Gemini API

4. PROJECT DESIGN

4.1 Problem Solution Fit

The Problem–Solution Fit maps each student problem to the EduGenie capability that addresses it.

Student Problem	EduGenie Solution	Endpoint	How it fits
Difficulty understanding complex concepts	Concept Explanation	/explain	Gives a simplified, step-by-step explanation with an example at the chosen level.
Need for quick clarification of doubts	Question Answering	/qa	Gives a concise answer with key points and a next step.
Difficulty revising long content	Summarization	/summarize	Condenses text into a summary, key points and important terms.
Difficulty assessing understanding	Quiz Generation	/quiz	Generates five multiple-choice questions with feedback for practice.
Unstructured self-learning	Learning Path	/learn/ recommendations	Provides ordered stages with topics, duration and resources.

EduGenie addresses these learning difficulties through a focused set of AI-assisted functions; it does not replace teachers or guarantee learning outcomes.

4.2 Proposed Solution

EduGenie provides one unified learning interface backed by a modular FastAPI application. Each learning function has its own endpoint and Python module. A module defines the instruction given to the AI model, the exact JSON structure expected in return, and a cleaning step that checks the response. The learner level (Beginner, Intermediate or Advanced) is added to the prompt so that language and depth match the learner. Responses are structured so that the interface can display them as cards, and the quiz is shown as an interactive question-by-question activity.

Module	Endpoint	Input	Structured output
qna.py	/qa	Academic question	Title, answer (under 150 words), 3–4 key points, example, next step
explanation_module.py	/explain	Concept / topic	Topic, simple explanation, 3–6 ordered steps, example, 3–4 key takeaways
quiz_module.py	/quiz	Topic or material	Topic and five multiple-choice questions, each with four options, correct answer and explanation
summary_module.py	/summarize	Passage of text	Title, summary, 3–6 key points, up to five important terms
learning_path.py	/learn/ recommendations	Topic	Topic, starting level, 4–6 stages (topics, description, duration), 3–5 resources

AI model selection: historical and current

During development the team explored a local model for Concept Explanation and later moved to a Gemini-based implementation. The history is summarised below; no benchmark values are claimed.

Stage	Description
Initial design	The initial project documentation planned Gemini (listed there as Gemini 1.5 Pro) for Q&A, summarization, quiz generation and learning paths, and the local LaMini-Flan-T5-783M model for concept explanation.
Local experiments	LaMini-Flan-T5-783M, LaMini-Flan-T5-248M and LaMini-Flan-T5-77M were tested in turn to find a local model with a practical response time.
Finding	Local model loading and inference were too slow for the required demonstration, so LaMini was not retained for the demonstration.
Gemini evaluation	Gemini 3.8 was considered and tested during model selection.
Current demonstration	Concept Explanation uses Gemini 3.5 . LaMini-Flan-T5 remains the locally tested, experimental implementation and is retained in the supplied source archive as such.

4.3 Solution Architecture

EduGenie follows a layered architecture. The presentation layer is the browser interface; the application layer is the FastAPI application that validates and routes requests; the feature modules implement the five functions; and the AI service layer is the Google Gemini API, an external cloud service. The locally tested LaMini-Flan-T5 model is shown as an experimental, non-active component.

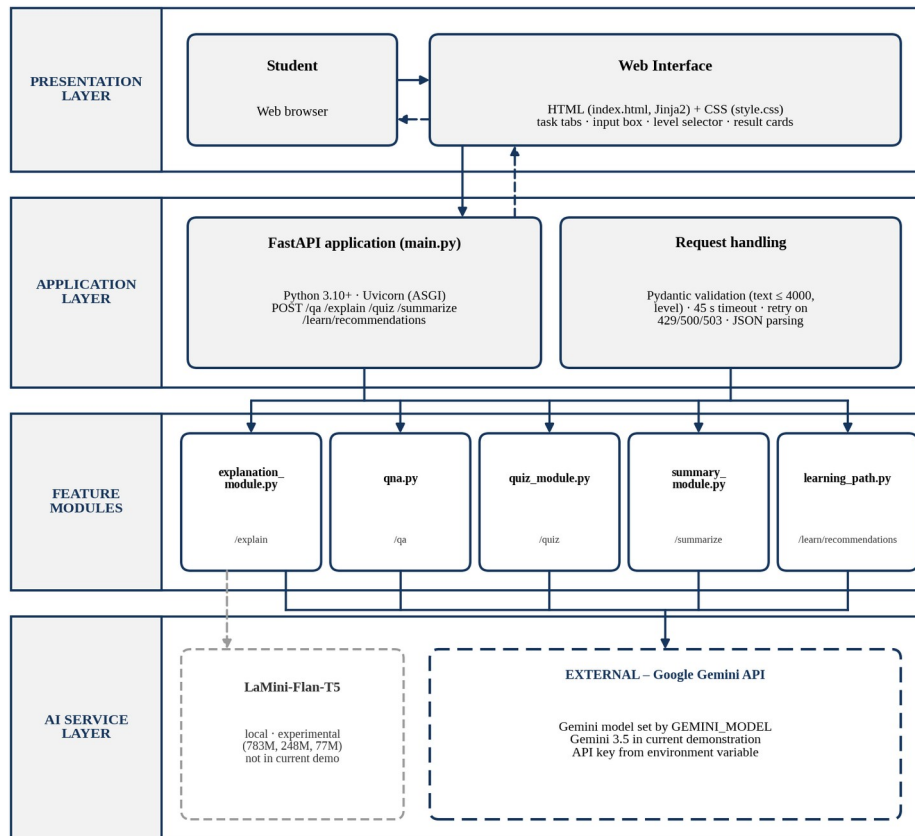


Fig. 4.1 – Solution architecture of EduGenie

Besides the five feature endpoints, the application serves the home page (GET /) and a readiness check (GET /health). There is no database, authentication or cloud deployment in the architecture.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project plan follows the Project Planning template (product backlog, sprint planning, user stories and story points). The work is planned as seven sprints of seven days each (49 days in total), from 01 October 2026 to 18 November 2026, with 15 user stories and 80 story points. The dates below are planned dates; actual completion details are not available in the provided project materials.

Product Backlog, Sprint Schedule and Estimation

Sprint	Functional Requirement (Epic)	User Story No.	User Story / Task	Story Points	Priority	Team Members
Sprint-1	Project Setup & Requirements	USN-1	As a student, I can use a system built from clearly documented requirements so that EduGenie addresses my real learning needs.	3	High	Mohammed Yasin, Dinesh Kumar
		USN-2	As a student, I can open EduGenie in my browser from a running FastAPI/Uvicorn server so that I can begin without extra setup.	5	High	Jayapriyan, Mohammed Yasin
Sprint-2	User Interface	USN-3	As a student, I can see a consistent Jinja2 page layout with header, navigation menu and home page so that I can move easily between modules.	5	High	Mathavan
	Backend & API	USN-4	As a student, I can submit input to a module URL and receive a structured response, with a clear message for empty input, so that results display correctly.	5	High	Jayapriyan
Sprint-3	AI Integration	USN-5	As a student, I can have my request processed by the local LaMini-Flan-T5-783M model so that responses use simple language.	5	High	Gokulnath
	Concept Explanation	USN-6	As a student, I can enter a difficult concept so that I receive a simplified explanation.	5	High	Gokulnath, Jayapriyan
		USN-7	As a student, I can read the explanation on a dedicated concept page, with a helpful message on failure, so that I can retry or rephrase.	3	Medium	Mathavan
Sprint-4	AI Integration	USN-8	As a student, I can get responses from the Google Gemini API, with secure key handling and clear failure messages, so that my session is not disrupted.	5	High	Gokulnath, Jayapriyan
	Question Answering	USN-9	As a student, I can submit a study question on a question page so that I receive a clear and relevant answer.	5	High	Jayapriyan, Mathavan
	Quiz Generation	USN-10	As a student, I can choose a topic and number of questions so that I get a quiz with multiple-choice options and correct answers for revision.	8	High	Gokulnath, Mathavan

Sprint	Functional Requirement (Epic)	User Story No.	User Story / Task	Story Points	Priority	Team Members
Sprint-5	Summarization	USN-11	As a student, I can paste educational text so that I receive a concise summary of it.	5	High	Gokulnath, Mathavan
	Learning Path	USN-12	As a student, I can enter a topic so that I receive a structured, step-by-step learning path to follow.	8	High	Gokulnath, Jayapriyan, Mathavan
Sprint-6	Integration & Testing	USN-13	As a student, I can switch between all five modules from one navigation menu so that I can use them in a single session.	5	High	Jayapriyan, Mathavan, Mohammed Yasin
		USN-14	As a student, I can use modules that passed functional testing and bug fixing so that every feature works as expected.	5	High	Dinesh Kumar, Mohammed Yasin
Sprint-7	Documentation & Final Submission	USN-15	As a student, I can rely on a final-tested, documented, demo-ready EduGenie (report, user manual, deployment preparation) so that its capabilities are clearly shown.	8	High	Dinesh Kumar, Mohammed Yasin

Note: USN-5 records the local LaMini-Flan-T5-783M integration as planned in the backlog; the model selection history is described in Section 4.2.

Project Tracker, Velocity and Burndown Chart

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed (as on Planned End Date)	Sprint Release Date (Actual)
Sprint-1	8	7 Days	01 Oct 2026	07 Oct 2026	Not available	Not available
Sprint-2	10	7 Days	08 Oct 2026	14 Oct 2026	Not available	Not available
Sprint-3	13	7 Days	15 Oct 2026	21 Oct 2026	Not available	Not available
Sprint-4	18	7 Days	22 Oct 2026	28 Oct 2026	Not available	Not available
Sprint-5	13	7 Days	29 Oct 2026	04 Nov 2026	Not available	Not available
Sprint-6	10	7 Days	05 Nov 2026	11 Nov 2026	Not available	Not available
Sprint-7	8	7 Days	12 Nov 2026	18 Nov 2026	Not available	Not available
Total	80	49 Days				

Not available = Not available in the provided project materials. The planning document leaves the story points completed and the actual sprint release dates blank.

Velocity: With a sprint duration of 7 days, the planned team velocity is 11.43 story points per sprint (80 story points over 7 sprints). The average velocity per day is $AV = \text{velocity} / \text{sprint duration} = 11.43 / 7 \approx 1.63$ story points per day.

Burndown Chart: A burndown chart is a graphical representation of work left to do versus time. The planned (ideal) burndown for EduGenie burns down the 80 story points evenly over 49 days (7 sprints of 7 days), as shown in Fig. 5.1. The actual burndown is not available in the provided project materials.

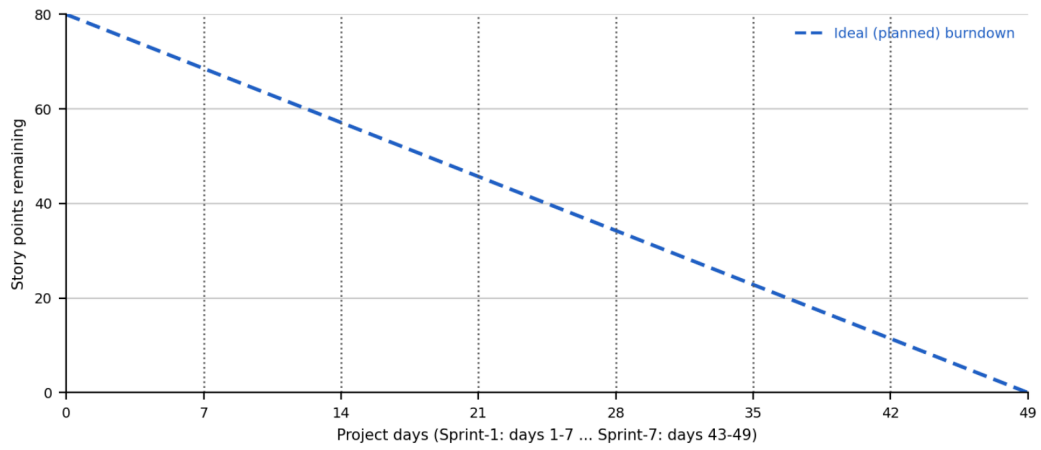


Fig. 5.1 – Planned (ideal) burndown chart of EduGenie

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Performance testing evaluates how the system behaves under expected and peak load conditions. For EduGenie, a load, concurrency and stress test plan using Apache JMeter was prepared, but the test plan has not been executed. No load-testing tool output or measured performance values were supplied in the project evidence, so none are reported. The manual functional use of the five functions through the web interface is shown in Section 7.1.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter (HTTP Request samplers, JSON body with text and level); planned, not yet executed
Type of Testing	Load Testing, Concurrency Testing and Stress Testing (planned)
Target Module / API	POST /qa, /quiz, /summarize, /learn/recommendations (Gemini API); POST /explain (local LaMini-Flan-T5-248M in the supplied source archive; the current demonstration uses Gemini 3.5)
Test Environment	Local: FastAPI app served by Uvicorn at 127.0.0.1:8000 (single process); JMeter on the same machine; Gemini API over the internet
Test Date	Pending: to be recorded when the JMeter test plan is executed

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Normal load: POST /qa and POST /summarize (Gemini), steady requests with 10 s ramp-up	5	120	HTTP 200 and success=true; avg and max response time compared with Step 3 targets
2	Concurrent load: mixed POST /qa, /quiz, /summarize and /learn/recommendations (Gemini)	10	180	All responses HTTP 200; error rate < 1%; no 504 timeouts
3	Higher AI-processing load: POST /explain on local LaMini-Flan-T5 (4 model generations per request)	3	120	HTTP 200; CPU and memory < 80%; response time recorded
4	Peak / stress load: mixed requests to all five POST endpoints, 60 s ramp-up	25	300	Graceful 429/503/504 handling and model fallback; error rate and limit recorded

The virtual users and durations above are the planned values of the JMeter test plan; none of these scenarios has been executed.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	Not measured	Pending	Requires JMeter run; depends on Gemini latency (and local T5 inference for /explain)
2	Response Time (Max)	< 5 seconds	Not measured	Pending	Requires JMeter run; app timeout is 45 s and retry back-off can add up to 8 s
3	Throughput (Req/sec)	–	Not measured	Pending	Requires JMeter run; limited by Gemini quota and CPU
4	Error Rate	< 1%	Not measured	Pending	Requires JMeter run; failures return HTTP 400/502/503/504
5	CPU Utilization	< 80%	Not measured	Pending	Requires OS monitor during the run, mainly for /explain
6	Memory Utilization	< 80%	Not measured	Pending	Requires OS monitor; T5 model stays loaded in the process

Target values are those given in the performance testing template; no Pass / Fail status is assigned because no measurements are available.

Step 4: Observations & Analysis

Key Findings: No performance test has been executed yet, so no measured result is claimed in this report. The findings below come from review of the source code and from the demonstration run.

- /qa, /quiz, /summarize and /learn/recommendations are async handlers that await Gemini through client.aio, so the event loop is not blocked while waiting.
- In the supplied source archive, /explain runs the local LaMini-Flan-T5-248M model via asyncio.to_thread and makes four sequential generations per request.
- Failures return JSON errors: 400 invalid input, 502 AI error, 503 busy/not configured, 504 timeout (45 s).
- During the demonstration run shown in Fig. 6.1, all five functions returned successful responses (HTTP 200 OK). The terminal log also shows warning entries for the model gemini-3.5-flash with HTTP 429 (rate-limit) responses and retry attempts before successful responses. This is an observation from the demonstration, not a measured performance result.

Bottlenecks Identified (expected from code review, not yet measured):

- Gemini latency and quota: on 429/500/503 the app retries twice per model with a 2 s sleep, then falls back from gemini-3.5-flash to gemini-3.1-flash-lite.
- /explain CPU load: concurrent requests share one process and CPU; the model loads lazily on the first call (cold start) without a lock.
- Deployment: single Uvicorn process (documented command: uvicorn main:app --reload); no caching or rate limiting.

Optimization Steps Taken: None applied yet (no test run). Already in the code: model fallback, retry with back-off, 45 s timeout and a 4000-character input limit. To try after the first run: warm up the T5 pipeline at startup, multiple Uvicorn workers and caching of repeated prompts. For the demonstration, Concept Explanation was moved from the local LaMini model to Gemini 3.5.

Note (documentation vs code): The project documentation mentions Gemini 1.5 Pro, LaMini-Flan-T5-783M and 3 quiz questions. The supplied code uses gemini-3.5-flash (fallback gemini-3.1-flash-lite), LaMini-Flan-T5-248M and 5 quiz questions. This section follows the code.

Step 5: Screenshots / Evidence

JMeter / Locust / k6 output is not available because the performance test has not been executed. The figures below are application evidence: Fig. 6.1 is from the demonstration run, and Fig. 6.2 is taken from the project documentation and shows the endpoints selected for testing.

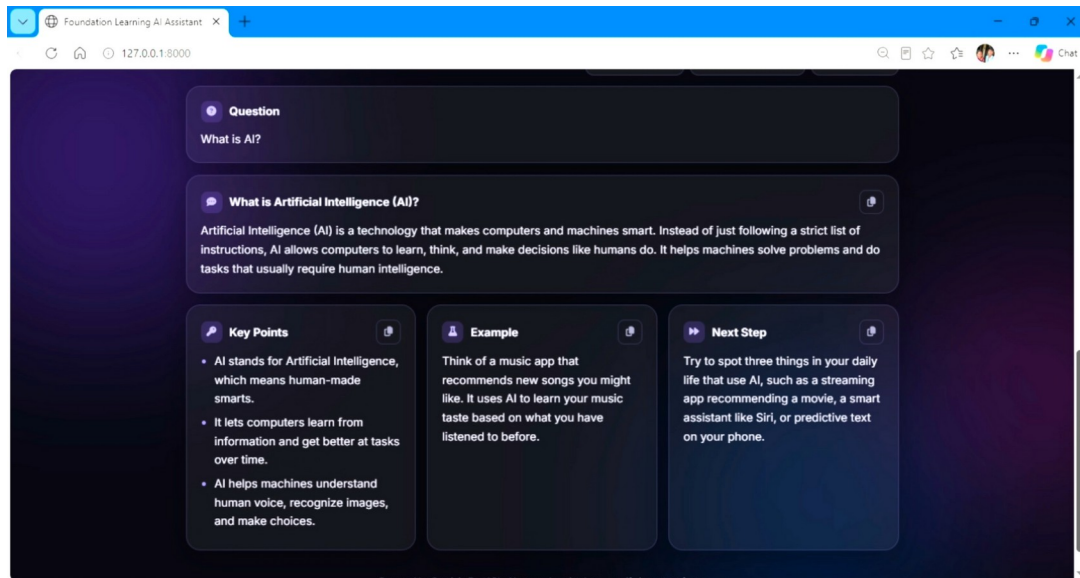


Fig. 6.1 – EduGenie project structure and Uvicorn terminal log showing 200 OK responses and 429 retry warnings

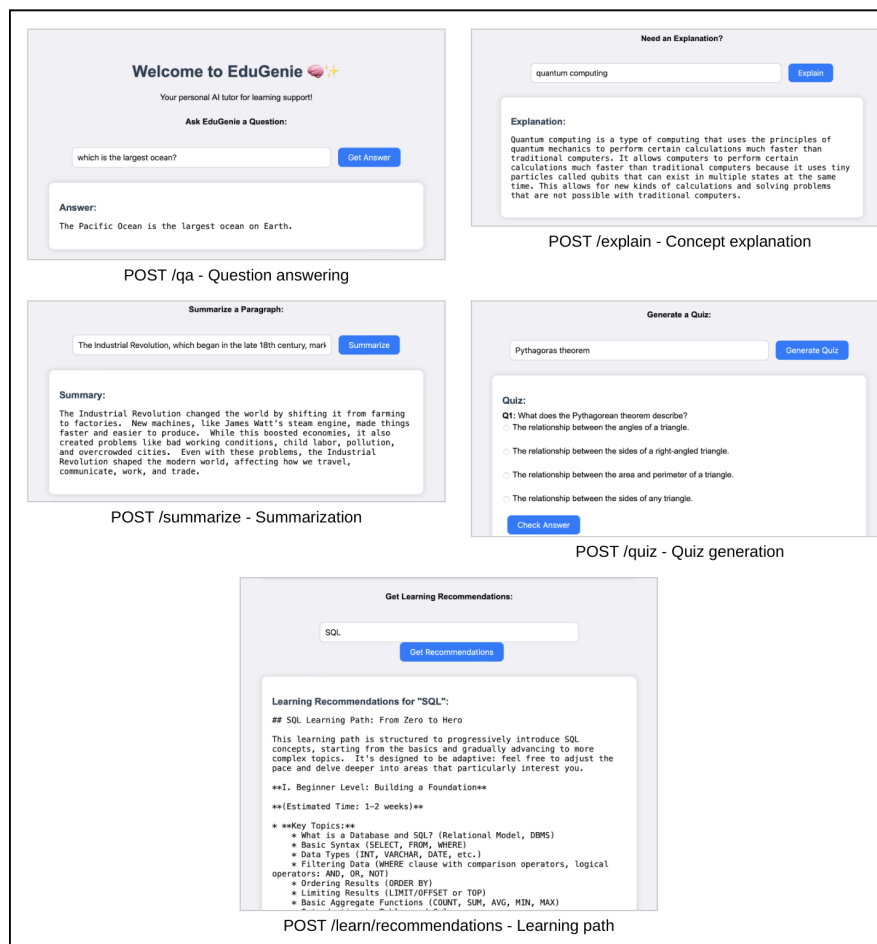


Fig. 6.2 – Application evidence for the endpoints selected for performance testing (from the project documentation)

6.2 User Acceptance Testing

User Acceptance Testing (UAT) confirms, with the intended users or stakeholders, that the delivered system meets their needs at the time of release. It is different from the manual functional use of the five functions shown in Section 7.1 and from the planned performance testing in Section 6.1; neither of these is reported as UAT.

Step 1: UAT Purpose and Scope

The purpose of the UAT document is to briefly explain the test coverage and open issues of the EduGenie project at the time of the release to User Acceptance Testing (UAT). The figures in Steps 2 and 3 are reproduced from the UAT Execution & Report Submission document dated 02 October 2026. Tester or stakeholder names, user feedback, ratings, acceptance percentage and approval details: Not available in the provided project materials.

Step 2: Defect Analysis

This analysis reports the number of resolved or closed bugs at each severity level and how they were resolved.

Resolution	Severity 1	Severity 2	Severity 3	Severity 4	Subtotal
By Design	0	1	1	2	4
Duplicate	0	0	1	1	2
External	1	1	0	0	2
Fixed	2	3	4	5	14
Not Reproduced	0	0	1	0	1
Skipped	0	0	0	1	1
Won't Fix	0	0	1	2	3
Totals	3	5	8	11	27

Step 3: Test Case Analysis

This analysis reports the number of test cases that have passed, failed and remain untested.

Section	Total Cases	Not Tested	Fail	Pass
Q&A Module	6	0	0	6
Explanation Module	6	0	0	6
Quiz Module	8	0	0	8
Summary Module	5	0	0	5
Learning Path Module	5	0	0	5
Frontend (Web Interface)	9	0	0	9
API & Error Handling	7	0	0	7

In the UAT document, no test case is recorded as Not Tested or Fail. Individual test case descriptions and any further results are not part of the UAT document: Not available in the provided project materials.

7. RESULTS

7.1 Output Screenshots

The following screenshots were captured from the running application at <http://127.0.0.1:8000> and show the actual output of each function. Screenshots are shown unmodified except for the removal of the empty black margins around the browser window.

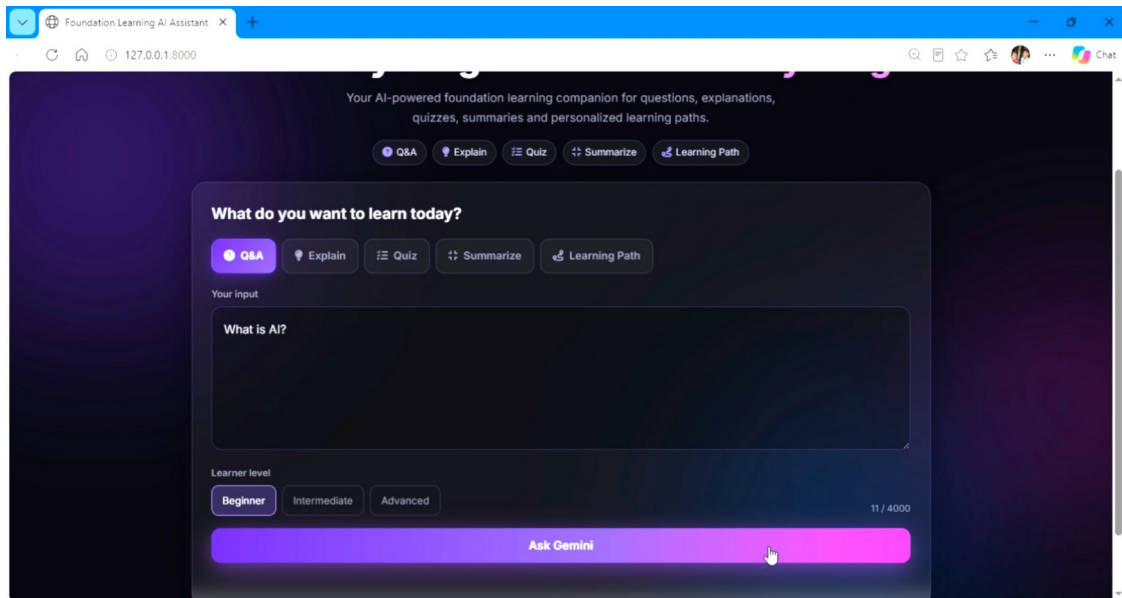


Figure 1 – EduGenie Main Interface and Q&A Input

The home page shows the five task buttons (Q&A, Explain, Quiz, Summarize, Learning Path), the input box with the question “What is AI?”, the learner level selector (Beginner selected), a character counter and the Ask Gemini button.

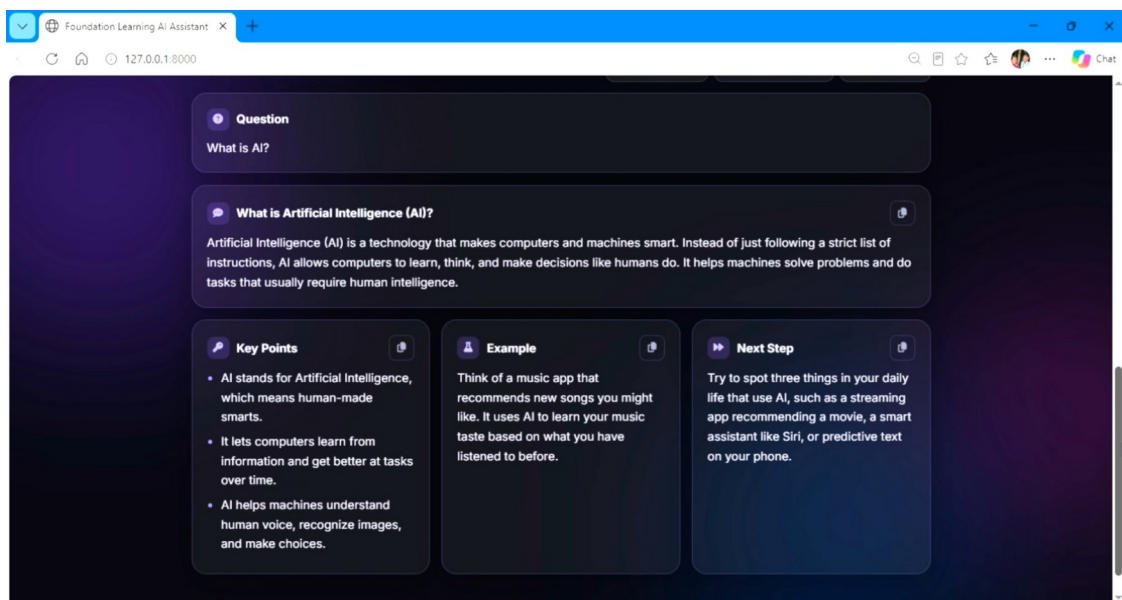


Figure 2 – Q&A Generated Response

The response shows the question, the generated answer and three further cards: Key Points, Example and Next Step, each with a copy button.

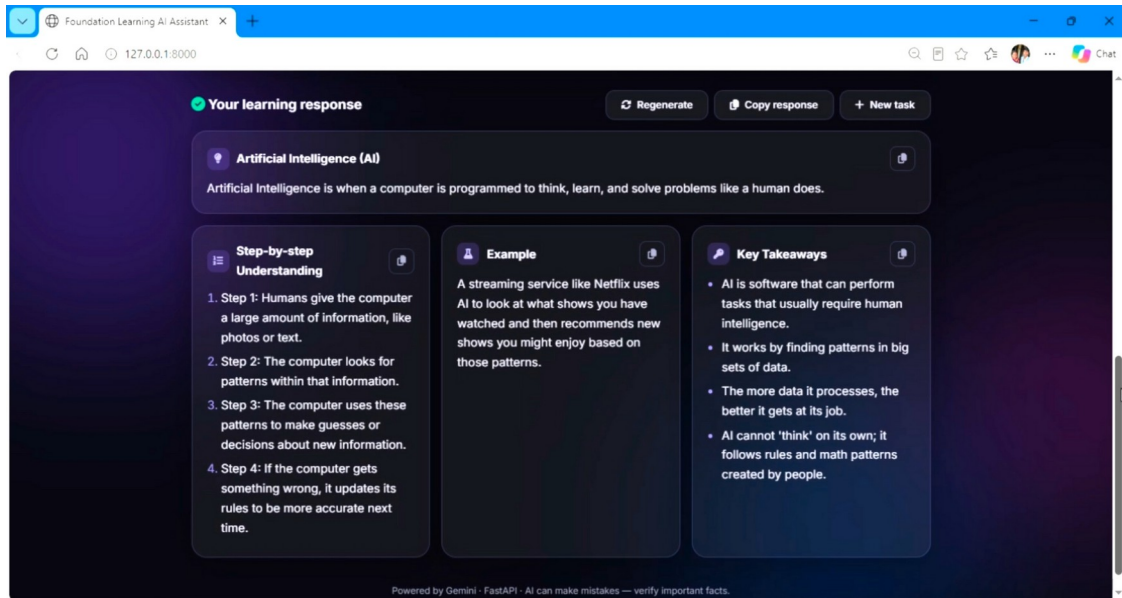


Figure 3 – Concept Explanation Output

The explanation of “Artificial Intelligence (AI)” is shown with Step-by-step Understanding, Example and Key Takeaways cards, together with the Regenerate, Copy response and New task buttons. This output was produced in the current demonstration configuration (Gemini 3.5).

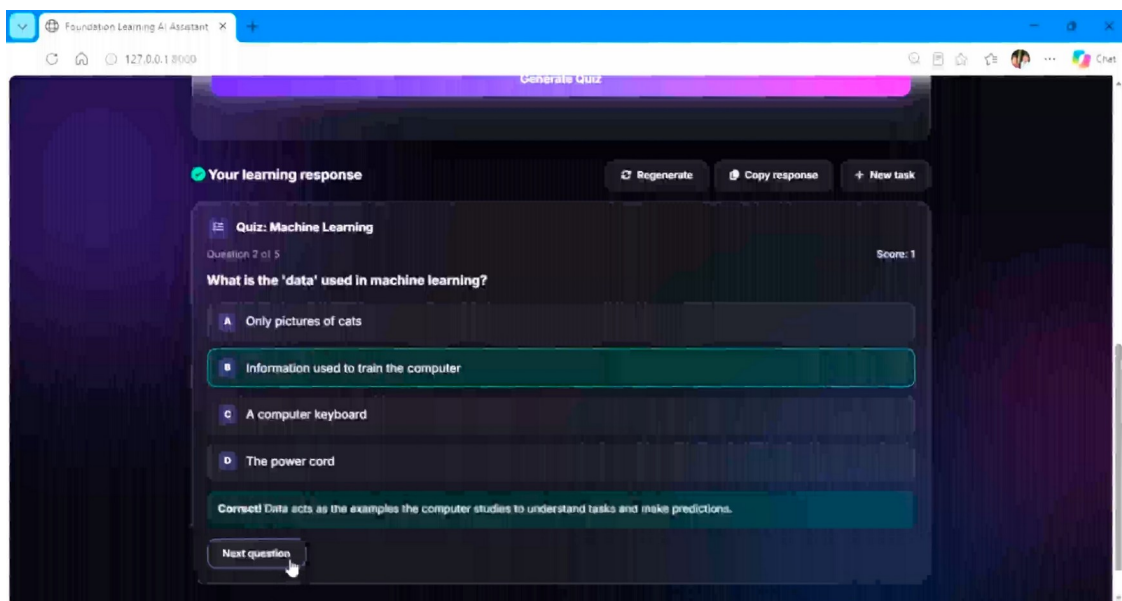


Figure 4 – Quiz Output

The quiz on “Machine Learning” is presented question by question (“Question 2 of 5”) with four options, a running score, a highlighted correct answer with a short explanation, and a Next question button.

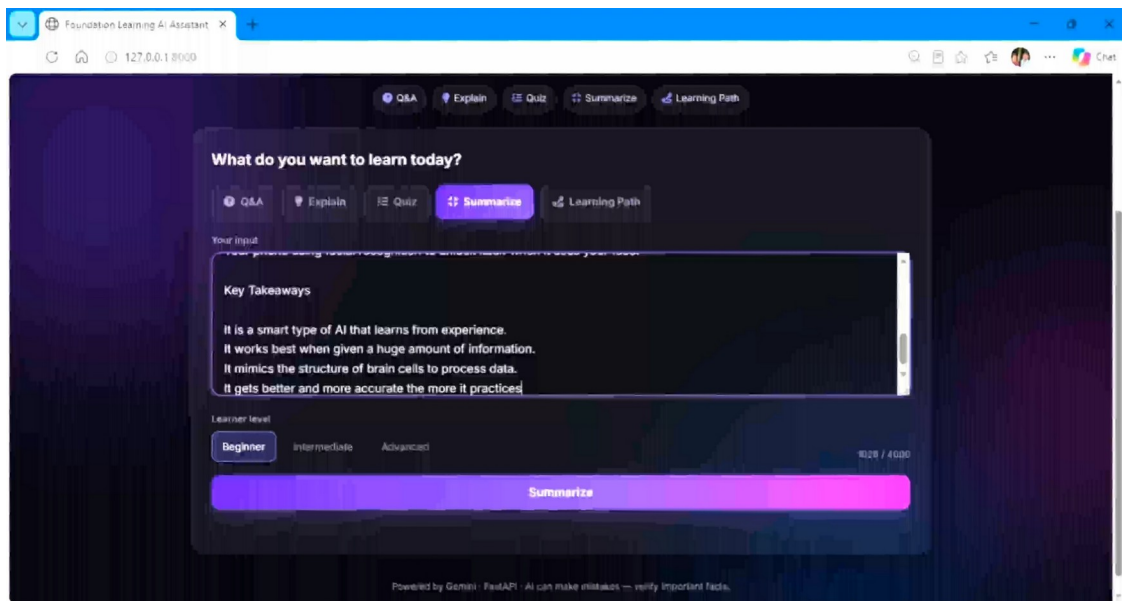


Figure 5 – Summarization Input Interface

The Summarize task is selected and a passage of learning text is entered in the input box, with the learner level selector, character counter and Summarize button visible.

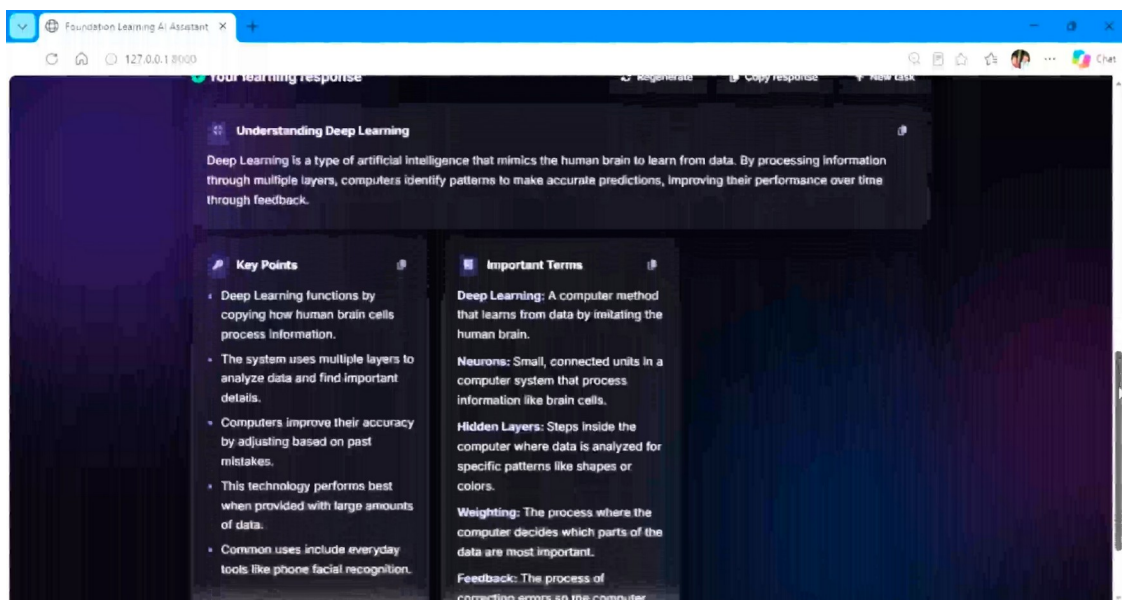


Figure 6 – Summarization Output

The result shows a concise summary titled “Understanding Deep Learning”, a Key Points card and an Important Terms card with short meanings.

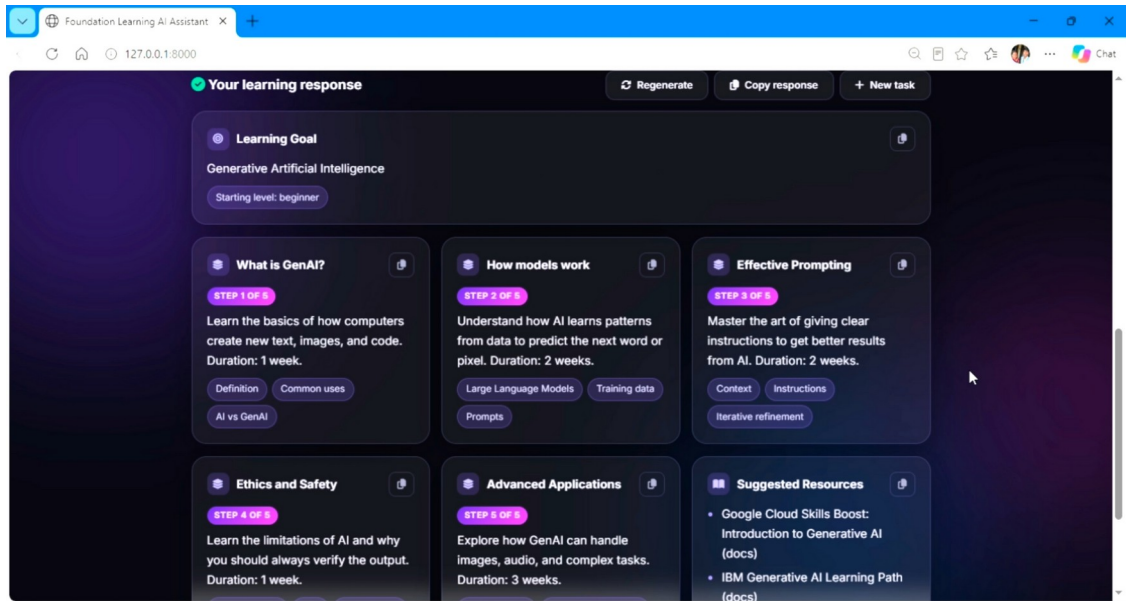


Figure 7 – Learning Path Output

The learning path for “Generative Artificial Intelligence” shows the learning goal, the starting level (beginner), five numbered steps with topics and durations, and a Suggested Resources card.

Screenshots of the Explain, Quiz and Learning Path input screens were not supplied and are therefore not included.

8. ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages
Five learning functions are available in one simple interface	AI-generated responses may contain errors and need verification
Modular backend: one module and endpoint per function	Gemini functions require a valid API key and internet connection
Responses are structured and displayed as readable cards	The application currently runs only locally; it is not deployed
Learner level (Beginner / Intermediate / Advanced) shapes the language and depth	Local LaMini-Flan-T5 models had practical response-time limitations
Interactive quiz with answer feedback and score	Response quality and speed depend on the model and API behaviour, including rate limits (HTTP 429)
Simple local execution with Uvicorn; no database needed	No saved history or progress between sessions

9. CONCLUSION

EduGenie addresses three common learning difficulties of students: understanding complex concepts and clearing doubts, following a structured learning progression, and reviewing and practising content. It does so through five functions – concept explanation, question answering, quiz generation, summarization and a learning path – available in one web interface.

The solution is built with a modular FastAPI backend, HTML/CSS/Jinja2 frontend and the Google Gemini API, with a separate module and endpoint for every function and structured, validated responses. During development the team learned that local LaMini-Flan-T5 models, although tested in three sizes, did not meet the response time needed for the demonstration, and the Gemini-based approach was therefore adopted; the current Concept Explanation demonstration uses Gemini 3.5.

In its current state EduGenie is a locally running academic prototype. Its output is AI-generated and should be verified, but it demonstrates how a single assistant can support explanation, clarification, revision, practice and learning guidance for students.

10. FUTURE SCOPE

The following enhancements are possibilities for future work. **None of them is implemented in the current version.**

- Voice interaction for hands-free learning
- Multilingual learning support
- A mobile application and offline capability
- Progress tracking and learning dashboards
- Gamification such as badges and learning streaks
- Adaptive learning paths based on learner performance
- Group study sessions and teacher / parent dashboards
- Integration with learning management systems (for example Moodle or Google Classroom)
- Image and PDF input for doubt solving and summarization
- Smart notifications
- Deployment beyond local execution

11. APPENDIX

Source Code

The complete source code is available in the project GitHub repository (see below). The project structure is:

File / folder	Purpose
main.py	FastAPI application: routing, validation, retry and error handling
explanation_module.py	Concept explanation logic
qna.py	Question answering
quiz_module.py	Quiz generation
summary_module.py	Summarization
learning_path.py	Learning recommendations
templates/index.html	Web frontend (Jinja2 template)
static/style.css	User interface styling
requirements.txt	Python dependencies

To run locally: install dependencies with `python -m pip install -r requirements.txt`, set the `GEMINI_API_KEY` environment variable, start the application with `uvicorn main:app --reload` and open `http://127.0.0.1:8000`.

Dataset Link

Not Applicable. No dataset was used; the application uses pre-trained AI models and the Google Gemini API.

GitHub & Project Demo Link

GitHub Repository: <https://github.com/Yasin87548/GenieAI.git>

Project Demo: <https://drive.google.com/drive/folders/1kQH0ztvga-0KiITzHTQDb7YhQkROiSig>